Day – 11 Decision tree Regressor ,Random forest Regressor and Logistic Regression

Core Concept of Decision Tree Regressor:

A Decision Tree Regressor is a type of supervised learning algorithm that uses a flowchart-like structure to predict a continuous target variable based on decision rules inferred from the data features. The model continuously splits data into subsets, based on the features that result in the lowest prediction error, forming a tree-like structure where:

- **Each internal node** represents a decision rule on a feature.
- **Each branch** represents the outcome of a decision.
- **Each leaf node** provides the predicted value (usually the average of values in that subset of data).

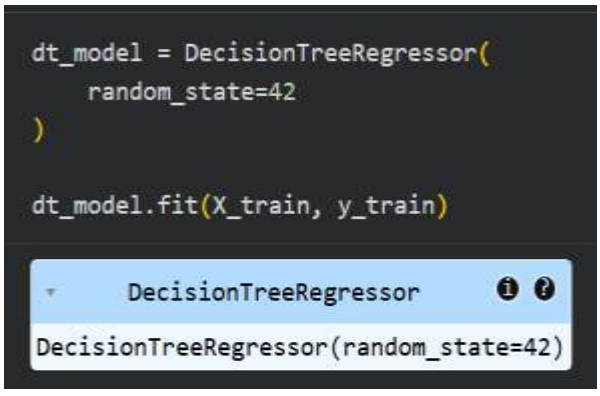
Decision Tree Regression is used for predicting continuous values effectively capturing non-linear patterns in data. Its tree-based structure makes model interpretability easy as we can tell why a decision was made and why we get this specific output. This information can further be used to fine tune model based on it flow of working.

Implementation:

Step 1 — Import Decision Tree Regressor

from sklearn.tree import DecisionTreeRegressor

Step 2 — Create and train the model



Step 3 — Make predictions

dt_pred = dt_model.predict(X_test)

Step 4 — Evaluate it

```
dt_mae = mean_absolute_error(y_test, dt_pred)
dt_mse = mean_squared_error(y_test, dt_pred)
dt_rmse = np.sqrt(dt_mse)
dt_r2 = r2_score(y_test, dt_pred)

print("MAE:", dt_mae)
print("MSE:", dt_mse)
print("RMSE:", dt_rmse)
print("R² Score:", dt_r2)
```

```
MAE: 5.746666666666667
MSE: 52.344027777777775
RMSE: 7.234917261294546
R² Score: 0.582051576186456
```

Step 5 — Visualize the Decision Tree 🌳

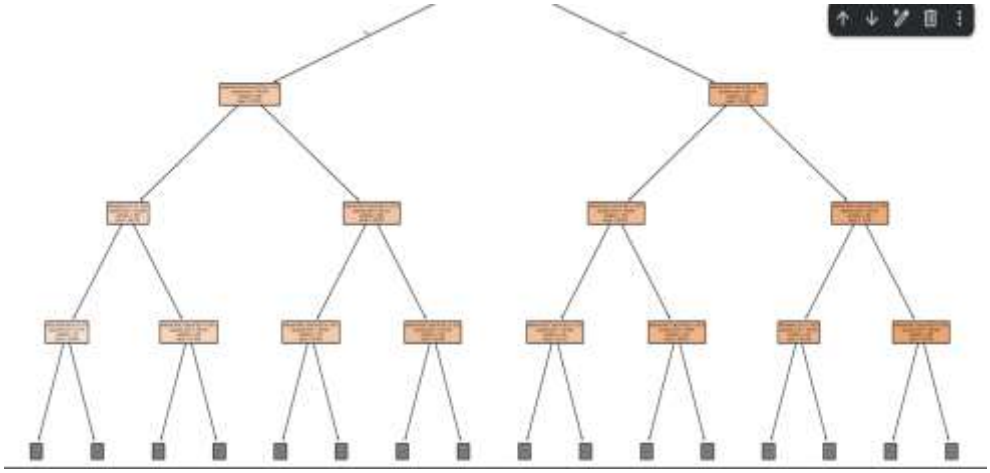
```
from sklearn.tree import plot_tree

plt.figure(figsize=(20, 10))

plot_tree(
    dt_model,
    feature_names=X.columns,
    filled=True,
    max_depth=3
)

plt.title("Decision Tree Regressor")

plt.show()
```



Random Forest Regressor:

Random Forest is an ensemble learning method that combines multiple decision trees to produce more accurate and stable predictions. It can be used for both classification and regression tasks, where regression predictions are obtained by averaging the outputs of several trees.

- Multiple decision trees: Builds many trees and combines their predictions.
- Ensemble approach: Reduces errors compared to a single decision tree.
- Regression prediction: Produces continuous values by averaging predictions from all trees.

Working of Random Forest Regression

Random Forest Regression works using the bagging (Bootstrap Aggregating) technique:

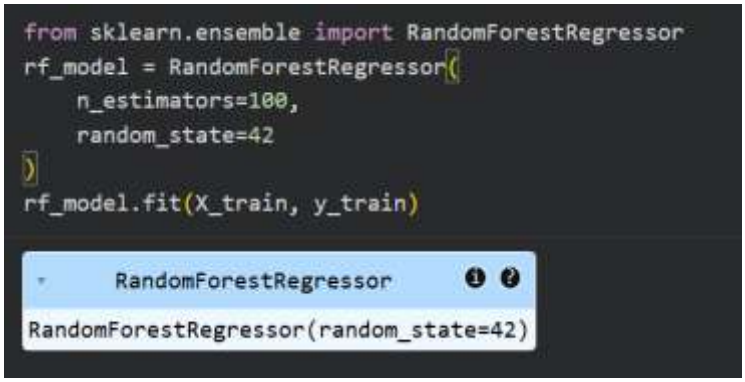
- Multiple decision trees are trained on different random subsets of the dataset with replacement to train each tree.
- Each tree uses a random subset of features while splitting nodes.
- Due to this each tree learns slightly different data and features, which increases model diversity.
- The final prediction is obtained by averaging the predictions from all decision trees.

Implementation:

Step 1 — Import Random Forest

from sklearn.ensemble import RandomForestRegressor

Step 2 — Create the Random Forest model



Step 3 — Train the model

rf_pred = rf_model.predict(X_test)

Step 4 — Make predictions

rf_pred = rf_model.predict(X_test)

Step 5 — Evaluate the model

```
rf_mae = mean_absolute_error(y_test, rf_pred)
rf_mse = mean_squared_error(y_test, rf_pred)
rf_rmse = np.sqrt(rf_mse)
rf_r2 = r2_score(y_test, rf_pred)
print("MAE:", rf_mae)
print("MSE:", rf_mse)
print("RMSE:", rf_rmse)
print("R² Score:", rf_r2)
```

```
MAE: 4.1602362698412705
MSE: 27.586356472780302
RMSE: 5.252271553602337
R² Score: 0.779732766161875
```

Random Forest prediction for a new student

```
new_student = pd.DataFrame({
    "Attendance (%)": [85],
    "Internal Test 1 (out of 40)": [30],
    "Internal Test 2 (out of 40)": [32],
    "Assignment Score (out of 10)": [8],
    "Daily Study Hours": [4]
})
prediction = rf_model.predict(new_student)
print("Predicted Final Exam Marks:", prediction[0])
```

```
Predicted Final Exam Marks: 65.5325
```

Logistic Regression:

Logistic Regression is a supervised machine learning algorithm used for classification problems. Unlike linear regression, which predicts continuous values it predicts the probability that an input belongs to a specific class.

- It is used for binary classification where the output can be one of two possible categories such as Yes/No, True/False or 0/1.
- It uses sigmoid function to convert inputs into a probability value between 0 and 1.

Assumptions:

1. **Independent Observations:** Each data point should be independent of the others, meaning there should be no dependence between observations.
2. **Binary Dependent Variable:** The target variable is typically binary and can take only two values, such as 0/1 or Yes/No. For multiclass problems, extensions such as Softmax-based logistic regression are used.
3. **Linearity of Log-Odds:** The independent variables should have a linear relationship with the log-odds of the dependent variable.

4. **No Extreme Outliers:** Extreme outliers can distort coefficient estimates and negatively affect model performance.
5. **Large Sample Size:** A sufficiently large dataset helps produce more reliable and stable predictions.

Understanding Sigmoid Function:

1. The sigmoid function is a key component of Logistic Regression that converts the model's raw output into a probability value between 0 and 1.

2. This function takes any real number and maps it into the range 0 to 1 forming an "S" shaped curve called the sigmoid curve or logistic curve. Because probabilities must lie between 0 and 1, the sigmoid function is perfect for this purpose.

3. In logistic regression, we use a threshold value usually 0.5 to decide the class label.

- If the sigmoid output is same or above the threshold, the input is classified as Class 1.
- If it is below the threshold, the input is classified as Class 0.

Implementation:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
df = pd.read_csv('/content/heart.csv')
print("Dataset Shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())
X = df.drop(columns=['target'])
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
y_pred_proba = model.predict_proba(X_test_scaled)[:, 1]
print("\n--- Model Performance ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.4f}\n")
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Dataset Shape: (1025, 14)

First 5 rows:
   age  sex  cp  trestbps  chol  fbs  restecg  thalach  exang  oldpeak  slope  \
0   52    1   0       125   212    0        1      168      0      1.0      2
1   53    1   0       140   203    1        0      155      1      3.1      0
2   70    1   0       145   174    0        1      125      1      2.6      0
3   61    1   0       148   203    0        1      161      0      0.0      2
4   62    0   0       138   294    1        1      106      0      1.9      1

   ca  thal  target
0   2     3       0
1   0     3       0
2   0     3       0
3   1     3       0
4   3     2       0

--- Model Performance ---
Accuracy: 0.8098

Confusion Matrix:
[[70 30]
 [ 9 96]]
```

```
Classification Report:
              precision    recall  f1-score   support

           0       0.89      0.70      0.78       100
           1       0.76      0.91      0.83       105

    accuracy                           0.81       205
   macro avg       0.82      0.81      0.81       205
weighted avg       0.82      0.81      0.81       205
```